

StockKeep

Shop Inventory & Stock Tracking System

Technical Debt Register & Repayment Plan

Technical_Debt_Plan.pdf — Standalone Document

Student Name:	Prince Ephraim Quarshie
Student ID:	22424601
Project Title:	StockKeep — AI Shop Inventory & Stock Tracking System
Institution:	University of Ghana
Department:	Computer Science
Module / Course:	CSCD602
Supervisor:	Prof. Solomon Mensah
Date:	13 August 2026

Every debt item below was identified either by direct source-code inspection or by live testing evidence in *Testing_Report.pdf* (defects DEF-01 to DEF-05), and is cross-referenced to the requirement it violates in *SRS.pdf*. None is speculative.

Contents

1	Introduction	2
2	Technical Debt Register	3
3	Technical Debt Repayment Roadmap	14
3.1	Immediate	14
3.2	Version 1.1	14
3.3	Version 2.0	14
3.4	Long-Term	15
4	Alignment with Future Evolution	16

1. Introduction

StockKeep’s git history shows a single initial commit (“AI-Powered StockKeep Tracker with Analytics Dashboard”) followed by rapid hardening commits addressing Vercel deployment, AI-route reliability, and several remediation passes. This pattern — a full-featured system built in a short, fixed window and then patched reactively — is the direct cause of most debt items below. An admin panel (role-based access, audit trail, user lock mode) was added in the most recent pass; it was security-reviewed before ever being committed, which is how TD-16 through TD-18 were caught pre-deployment rather than post-incident. In total, **eight items (TD-01, TD-02, TD-03, TD-06, TD-13, TD-14, TD-16, TD-17, TD-18, plus SRS.pdf’s NFR-02) have been fully resolved, and TD-08 has been mitigated** (live functionality restored, though not yet made fully durable) — all verified against a production build and, for TD-08 and the admin panel, against the actual live deployment. See the Status column and `Testing_Report.pdf` for live evidence. The remaining items are recorded honestly as still open, with a prioritised, realistic plan to repay them.

2. Technical Debt Register

Debt ID	Technical Debt	Cause	Impact	Priority	Status
TD-01	Login checked a hard-coded/env passcode (APP_PASSCODE, default 1234), never the Settings screen let a user edit.	Architecture shortcut - authentication was wired directly to an env constant early on, and the Settings screen added a passcode field to the database without updating the login route to read it.	Store staff who changed the passcode in Settings believed they had changed it, but the old passcode still worked and the new one did nothing.	Critical	RESOLVED. src/lib/passcode.ts (scrypt hashing) + login route now reads Settings.passcode as the source of truth. Verified live: old passcode rejected, new passcode accepted (Testing_Report.pdf V-02, V-03).

Debt ID	Technical Debt	Cause	Impact	Priority	Status
TD-02	POST <code>/api/seed?force=true</code> was reachable without authentication and deleted all <code>Item</code> , <code>Supplier</code> , <code>Sale</code> , and <code>StockMovement</code> records before reseeding. Confirmed exploitable live (<code>Testing_Report.pdf</code> , DEF-02).	Convenience shortcut to guarantee a populated demo on first Vercel request, whitelisted broadly in <code>middleware.ts</code> rather than narrowly.	Any unauthenticated actor who discovered the endpoint could wipe production data.	Critical	RESOLVED, then hardened further. Removed <code>/api/seed</code> from the middleware's public whitelist (verified live: unauthenticated call now returns 401, V-01). In a follow-up pass the <code>force/wipe</code> capability was removed from <code>src/lib/seed.ts</code> and <code>src/app/api/seed/route.ts</code> entirely - the endpoint can no longer delete data under any circumstances, authenticated or not. The Settings screen's "Demo Data Generator" button, the only UI that ever called <code>force=true</code> , was removed at the same time; the system now relies solely on the existing non-destructive auto-populate-if-empty behaviour.

Debt ID	Technical Debt	Cause	Impact	Priority	Status
TD-03	No rate limiting, delay, or lockout on <code>/api/auth/login</code> . Confirmed live - 5 rapid wrong-passcode attempts all returned instantly (DEF-03).	Authentication simplification - a single shared 4-digit passcode was treated as “good enough” for the project’s time budget, without also budgeting for brute-force protection.	An attacker could brute-force a 4-digit numeric passcode in well under a minute at typical network speeds.	High	RESOLVED. <code>src/lib/rateLimit.ts</code> adds a 5-attempts/5-minutes per-IP limiter. Verified live: attempts 5-6 return 429 (V-10). Documented limitation: in-memory, per-process only - not shared across multiple serverless instances.
TD-04	No CSRF token on state-changing routes; protection relies solely on the <code>SameSite=Lax</code> cookie attribute.	Authentication simplification, same root cause as TD-03.	Reduced but non-zero cross-site request forgery exposure on POST/PUT/DELETE routes.	Medium	Open - Scheduled for Future Resolution
TD-05	No security response headers configured (<code>next.config.ts</code> has no <code>headers()</code> - no CSP, HSTS, X-Frame-Options). Confirmed live (Testing_Report TC-API-26).	Architecture shortcut - the default Next.js scaffold was never hardened post-scaffold.	Weaker defence-in-depth against clickjacking/XSS/protocol-downgrade in a production deployment.	Medium	Open - Scheduled for Future Resolution

Debt ID	Technical Debt	Cause	Impact	Priority	Status
TD-06	The Resend email API key was stored in plaintext in the <code>Settings</code> database table and returned verbatim by <code>GET /api/settings</code> to any authenticated caller (confirmed live, DEF-05).	Dependency/configuration choice - third-party secret was modelled as ordinary per-store configuration data rather than a protected secret.	Any unauthenticated session (i.e., anyone with the shared passcode) could read the shop's real Resend API key.	High	RESOLVED. <code>GET /api/settings</code> now returns <code>resendApiKeyConfigured</code> as a boolean instead of raw values; the Settings UI's key/passcode fields are write-only. Verified live (V-04).
TD-07	Zero automated tests (no unit, integration, or E2E suite; no test script in <code>package.json</code>).	Limited automated testing - deliberately deferred under the project's time constraint in favour of shipping features.	No regression safety net; every change risks silently breaking existing behaviour (as DEF-01 in <code>Testing_Report.pdf</code> demonstrated was already possible).	High	Open - Scheduled for Future Resolution

Debt ID	Technical Debt	Cause	Impact	Priority	Status
TD-08	SQLite (prisma/dev.db, a local file) is used as the database provider while the stated deployment target is Vercel serverless, which has an ephemeral, largely read-only filesystem outside /tmp. Confirmed live: before mitigation, every data route on the deployed site (items, analytics, settings) returned HTTP 500 - this had never reproduced in local testing and was only caught by testing the actual live URL.	Database/infrastructure decision made for local-development simplicity, not revisited for the production target.	The live site was non-functional for all data operations (read and write) prior to mitigation.	Critical	MITIGATED, not fully resolved. src/lib/prisma.ts now copies dev.db into /tmp (the one writable path on Vercel) on cold start and points Prisma there (next.config.ts outputFileTracingIncludes ensures the file is bundled). Verified live: reads and writes both work again (Supporting_Files/Test_Ev V-13-V-16). This is a stopgap, not a fix - /tmp is ephemeral, so data can still be lost on a cold start or land on a different function instance. A real managed database (Vercel Postgres/Neon/Turso) is still needed for durable production data and remains the top open priority.

Debt ID	Technical Debt	Cause	Impact	Priority	Status
TD-09	The frontend is a single 1,921-line client component (<code>src/app/page.tsx</code>) handling nine views, all modals, and all state, with no component decomposition.	Time-constrained delivery - building one file was faster initially than setting up a component architecture.	Harder to test, harder to onboard a second developer to.	Medium	Open - Scheduled for Future Resolution
TD-10	<code>items</code> GET fetches the entire table into memory and filters/search in JavaScript rather than pushing filters into the Prisma query; <code>sales</code> GET hardcodes <code>take: 100</code> with no pagination controls.	Incomplete refactoring - filtering was implemented against an in-memory array early on and never migrated to query-level filtering as the dataset grew.	Will not scale past a small inventory.	Medium	Open - Scheduled for Future Resolution
TD-11	<code>insightsCache</code> (5-minute AI insights cache) is a plain module-level JavaScript variable.	Temporary solution - an in-memory cache is the simplest possible implementation.	On Vercel serverless, each function instance has its own memory, so the “5-minute cache” does not behave consistently in production.	Low	Open - Acceptable Temporarily

Debt ID	Technical Debt	Cause	Impact	Priority	Status
TD-12	Several catch blocks silently swallow errors - failures are logged server-side only and never surfaced to the user.	Error handling was scoped down to “don’t let a side-effect break the main request” without also budgeting for user-facing visibility.	A shop owner relying on low-stock email alerts (FR-14) may never realise the emails have silently stopped sending.	Medium	Open - Scheduled for Future Resolution
TD-13	The Gemini model name (gemini-1.5-flash) was hardcoded in src/lib/gemini and returned 404 Not Found from the real API - every AI feature silently ran on its fallback path only (DEF-04).	Dependency choice not revisited - the model name was pinned once and never re-validated against the provider’s current supported-model list; the provider retired the model between when the code was written and now.	The “AI-powered” features were not producing genuine AI-generated output in practice.	High	RESOLVED. Tested 5 candidate models live against the real API; switched to gemini-flash-latest (a Google-maintained alias, chosen specifically to resist future retirements). Verified live: genuine, non-templated AI responses returned (V-08, V-09).

Debt ID	Technical Debt	Cause	Impact	Priority	Status
TD-14	<code>README.md</code> was originally the unmodified default <code>create-next-app</code> boilerplate.	Documentation gap - normal for a time-boxed build, but never revisited afterward.	A new developer (or the marker) had no in-repo entry point beyond source reading.	Low	RESOLVED. <code>README.md</code> rewritten with real project overview, feature list, tech stack, database schema, full API route table, local setup steps, and links to this documentation package. Reviewed for factual accuracy in the same pass as TD-18 below.
TD-15	Several route handlers type caught errors as <code>catch (error: any)</code> , opting out of the strict TypeScript checking enabled elsewhere (<code>tsconfig.json</code> , <code>"strict": true</code>).	Convenience shortcut around TypeScript's stricter error-typing ergonomics.	Slightly weakens type-safety guarantees; low real-world impact.	Low	Open - Acceptable Temporarily

Debt ID	Technical Debt	Cause	Impact	Priority	Status
TD-16	(Found in code review, never shipped) The initial admin-panel implementation's <code>/api/auth/admin/login</code> fell back to comparing the submitted passcode against a hardcoded plaintext constant (<code>DEFAULT_ADMIN_PASSCODE</code>) whenever <code>Settings.adminPasscode</code> was unset - which it always was until an admin explicitly changed it via the UI. That constant's value was also printed directly in <code>README.md</code> (see TD-18), so the exposure would have been trivial to find.	Copy-paste from the regular-login bootstrap pattern without carrying over the <code>then-only-ever-compare-hashes</code> discipline that depends on.	Full administrative access (lock-mode toggle, audit log, session history, admin passcode change) for any attacker who read the public source, until an admin manually changed the default - which nothing in the UI prompted them to do urgently.	Critical (would have been, had it shipped)	RESOLVED before this code was ever committed. <code>src/app/api/auth/admin-login</code> and <code>src/app/api/admin/settings</code> now self-heal: on first use, the default is hashed and persisted to <code>Settings.adminPasscode</code> immediately, and every subsequent check goes through <code>verifyPasscode</code> (scrypt) like the regular passcode. No plaintext comparison path remains. Verified locally and live: the old default is rejected (<code>Testing_Report.pdf</code>, admin panel review pass).

Debt ID	Technical Debt	Cause	Impact	Priority	Status
TD-17	(Found in code review, never shipped) The same admin-login route's Settings-bootstrap branch (used only if no Settings row exists at all) hashed the <i>admin</i> default passcode into the <i>regular user</i> passcode column, instead of adminPasscode.	Copy-paste error - the bootstrap block was adapted from the regular login route without updating which field the hash was written to.	On a fresh deployment where the admin-login endpoint happened to be hit before any other route, the regular shared passcode would have silently become the admin default instead of the intended user default - locking out or confusing regular users.	High (would have been, had it shipped)	RESOLVED before this code was ever committed. Each default is now hashed into its own column in the bootstrap branch.

Debt ID	Technical Debt	Cause	Impact	Priority	Status
TD-18	(Found in code review, never shipped) <code>README.md</code> (as first drafted for the admin panel) stated the admin passcode was hashed with <code>bcrypt</code> (the codebase has no <code>bcrypt</code> dependency; it uses Node's built-in <code>script</code>, same as the regular passcode), printed the actual default admin passcode value directly in a public file, and stated an outdated <code>Next.js</code> version (15, vs. the actual 16.3.0) and Gemini model name.	Documentation written from assumption/template rather than verified against the actual code and <code>package.json</code>.	A reader (examiner or future maintainer) would be misled about the hashing algorithm, and the credential exposure compounds TD-16 by making the vulnerable default easy to find.	Medium	RESOLVED. <code>README</code> corrected on all four points; credential values removed from <code>README</code> entirely and centralised in <code>Deployment_and_Source_Li</code> only, consistent with how the regular passcode is already handled (see the earlier passcode-hint removal from the login page).

3. Technical Debt Repayment Roadmap

3.1 Immediate

- **TD-01** — Make login check `Settings.passcode` (hashed) instead of `/` in addition to `APP_PASSCODE`. **DONE.**
- **TD-02** — Remove `/api/seed` from the unauthenticated middleware whitelist. **DONE.**
- **TD-03** — Add basic login attempt throttling. **DONE.**
- **TD-06** — Move the Resend API key handling so it is never returned to the client; stop returning it from `GET /api/settings`. **DONE.**
- **TD-13** — Confirm which Gemini model the configured API key actually has access to and update `src/lib/gemini.ts`. **DONE.**
- **TD-08** — **was completely broken live. MITIGATED.** A `/tmp-copy` stopgap (see register above) restored full read/write functionality on the live deployment, verified live. **Still recommended before real shop data is trusted to it:** migrate to a real managed database (Vercel Postgres / Neon / Turso) for durable persistence, since `/tmp` remains ephemeral. This is the one remaining Immediate-priority item requiring an infrastructure decision (provisioning a database, setting a `DATABASE_URL` on Vercel) rather than a self-contained code change, so it was appropriately scoped as “make it work now, make it durable next” rather than left broken.

3.2 Version 1.1

Next planned iteration, once Immediate items are resolved:

- **TD-04** — Add CSRF tokens (or migrate fully to a same-site-only architecture with `SameSite=Strict` where UX allows) on all state-changing routes.
- **TD-05** — Add security headers via `next.config.ts headers()` (CSP, X-Frame-Options, Referrer-Policy, HSTS on the production domain).
- **TD-07** — Introduce an automated test suite (Vitest or Jest for unit tests on `src/lib/validations.ts` and `src/lib/seed.ts`; Playwright for the core login → add item → adjust stock → sale flow), starting from the 26 manual test cases already documented in `Testing_Report.pdf` so existing coverage is not lost, only automated.
- **TD-12** — Surface currently-silent failures (seed trigger, low-stock email) to the UI or an admin-visible log, rather than only the server console.

3.3 Version 2.0

Structural work once the product direction is validated:

- **TD-09** — Decompose `src/app/page.tsx` into a proper `src/components/` tree (one component per view/modal), enabling code-splitting and independent testing.
- **TD-10** — Move filtering/search/pagination for `items` and `sales` into the Prisma query layer (`where`, `skip/take`) and expose pagination controls in the UI.

- **TD-11** — Replace the in-memory `insightsCache` with a shared cache appropriate to the serverless target (e.g., Vercel KV/Redis) if the 5-minute caching behaviour is still desired at scale.

3.4 Long-Term

Dependent on real usage/scale, revisited after Version 2.0 ships:

- Introduce genuine multi-user accounts and role-based access control if the single-shared-passcode model no longer fits the shop's needs (this is a product decision, not only a technical one).
- Reassess SQLite-vs-managed-database (TD-08) again in light of actual measured usage once on a durable backend, to confirm the chosen database still fits the read/write pattern.
- **TD-14, TD-15** — Backfill project documentation into the repository itself (a real `README.md`, an `ARCHITECTURE.md`) and clean up remaining `catch (error: any)` typing as the codebase stabilises.

4. Alignment with Future Evolution

The roadmap above is sequenced so that every “Immediate” item removes a risk that would otherwise undermine trust in the platform (data loss, credential exposure, unauthenticated destructive actions) before any feature work continues. Version 1.1 turns the ad-hoc manual testing and security posture established during this documentation pass into durable, automated safeguards. Version 2.0 pays down the structural (frontend architecture, query-layer) debt that would otherwise make every future feature progressively more expensive to add. This ordering is deliberate: it is cheaper to restructure `page.tsx` (TD-09) once the underlying auth/data risks (TD-01, TD-02, TD-08) are no longer in flux, since those fixes will themselves touch several of the same routes.